HDOC DAY – 41

ALGORITHM:

1. Start.
2. Declare two character arrays for two paragraphs.
3. Read Paragraph 1 using readParagraph().
4. Read Paragraph 2 using readParagraph().
5. Display the menu:
    - Find longest and shortest words.
    - Remove repeated consecutive spaces.
    - Convert paragraphs to title case.
    - Find frequency of distinct words.
    - Exit.
6. Read the user's choice.
7. If choice is 1:
    - Copy the paragraph into a temporary string.
    - Use strtok() to divide it into words.
    - Consider the first word as both longest and shortest initially.
    - Compare the length of every word using strlen().
    - Store a word using strcpy() whenever a longer or shorter word is found.
    - Repeat for both paragraphs.
    - Display the longest and shortest words.
8. If choice is 2:
    - Scan each paragraph character by character.
    - Use isspace() to identify spaces.
    - Copy the first space to the modified paragraph.
    - Ignore additional consecutive spaces.
    - Copy all non-space characters normally.
    - Display both modified paragraphs.

9. If choice is 3:
    - Scan each paragraph.
    - Identify the beginning of every word.
    - Convert the first alphabet of every word to uppercase using toupper().
    - Convert the remaining alphabets to lowercase using tolower().
    - Display both paragraphs in title case.
10. If choice is 4:
    - Convert a copy of the paragraph to lowercase.
    - Divide it into words using strtok().
    - Store each distinct word in an array.
    - Compare words using strcmp().
    - If a word already exists, increase its frequency.
    - Otherwise, add it as a new word with frequency 1.
    - Repeat for both paragraphs.
    - Display every distinct word and its frequency.
11. If choice is 5, exit.
12. For any other choice, display an invalid-choice message.
13. Repeat the menu until Exit is selected.
14. Stop.

C PROGRAM:

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


#define MAX 1000

#define WORD 100

#define MAX_WORDS 200
```

```
// Function prototypes
void readParagraph(char paragraph[], size_t size);

void findLongestAndShortestWords(const char paragraph[],
                                 char longestWord[],
                                 char shortestWord[]);

void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[]);

void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[]);

void findWordFrequencies(const char paragraph[]);



// 1. Read a paragraph
void readParagraph(char paragraph[], size_t size)
{
   fgets(paragraph, size, stdin);

   // Remove newline added by fgets()
   paragraph[strcspn(paragraph, "\n")] = '\0';
}



// 2. Find longest and shortest words
void findLongestAndShortestWords(const char paragraph[],
```

```
                    char longestWord[],
                    char shortestWord[])
{
   char temp[MAX];
   char *word;

   strcpy(temp, paragraph);

   word = strtok(temp, " ,.!?;:\t\n");

   if (word == NULL)
   {
      strcpy(longestWord, "");
      strcpy(shortestWord, "");
      return;
   }

   // First word is initially both longest and shortest
   strcpy(longestWord, word);
   strcpy(shortestWord, word);

   while (word != NULL)
   {
      if (strlen(word) > strlen(longestWord))
      {
         strcpy(longestWord, word);
      }
```

```
      if (strlen(word) < strlen(shortestWord))
      {
         strcpy(shortestWord, word);
      }

      word = strtok(NULL, " ,.!?;:\t\n");
   }
}


// 3. Remove repeated consecutive spaces
void removeRepeatedSpaces(const char paragraph[],
                          char modifiedParagraph[])
{
   int i = 0;
   int j = 0;
   int previousSpace = 0;

   while (paragraph[i] != '\0')
   {
      if (isspace((unsigned char)paragraph[i]))
      {
         // Add only one space
         if (!previousSpace)
         {
            modifiedParagraph[j] = ' ';
            j++;
            previousSpace = 1;
```

```
        }
      }
      else
      {
        modifiedParagraph[j] = paragraph[i];
        j++;
        previousSpace = 0;
      }

      i++;
    }

    modifiedParagraph[j] = '\0';
}


// 4. Convert paragraph to title case
void convertToTitleCase(const char paragraph[],
                        char titleCaseParagraph[])
{
    int i = 0;
    int newWord = 1;

    while (paragraph[i] != '\0')
    {
      if (isalpha((unsigned char)paragraph[i]))
      {
        if (newWord)
```

```
        {
            titleCaseParagraph[i] =
                toupper((unsigned char)paragraph[i]);

            newWord = 0;
        }
        else
        {
            titleCaseParagraph[i] =
                tolower((unsigned char)paragraph[i]);
        }
    }
    else
    {
        titleCaseParagraph[i] = paragraph[i];

        if (isspace((unsigned char)paragraph[i]))
        {
            newWord = 1;
        }
    }

    i++;
  }

  titleCaseParagraph[i] = '\0';
}
```

```
// 5. Find frequency of every distinct word
void findWordFrequencies(const char paragraph[])
{
   char temp[MAX];
   char words[MAX_WORDS][WORD];
   int frequency[MAX_WORDS] = {0};

   char *word;
   int wordCount = 0;
   int i, j;
   int found;

   strcpy(temp, paragraph);

   // Convert paragraph to lowercase
   for (i = 0; temp[i] != '\0'; i++)
   {
      temp[i] = tolower((unsigned char)temp[i]);
   }

   word = strtok(temp, " ,.!?;:\t\n");

   while (word != NULL)
   {
      found = 0;

      // Check if word already exists
```

```
        for (i = 0; i < wordCount; i++)
        {
            if (strcmp(words[i], word) == 0)
            {
                frequency[i]++;
                found = 1;
                break;
            }
        }

        // New word
        if (!found && wordCount < MAX_WORDS)
        {
            strcpy(words[wordCount], word);
            frequency[wordCount] = 1;
            wordCount++;
        }

        word = strtok(NULL, " ,.!?;:\t\n");
    }

    printf("\nWord Frequencies:\n");

    for (j = 0; j < wordCount; j++)
    {
        printf("%s : %d\n", words[j], frequency[j]);
    }
}
```

```
// Main function
int main()
{
    char paragraph1[MAX];
    char paragraph2[MAX];

    char longest1[WORD], shortest1[WORD];
    char longest2[WORD], shortest2[WORD];

    char modified1[MAX], modified2[MAX];
    char title1[MAX], title2[MAX];

    int choice;

    // Read both paragraphs
    printf("Enter Paragraph 1:\n");
    readParagraph(paragraph1, MAX);

    printf("\nEnter Paragraph 2:\n");
    readParagraph(paragraph2, MAX);

    do
    {
        printf("\n========== MENU ==========\n");
        printf("1. Find longest and shortest words\n");
        printf("2. Remove repeated consecutive spaces\n");
```

```
printf("3. Convert paragraphs to title case\n");
printf("4. Find word frequencies\n");
printf("5. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice)
{
   case 1:

      findLongestAndShortestWords(
         paragraph1, longest1, shortest1);

      findLongestAndShortestWords(
         paragraph2, longest2, shortest2);

      printf("\nParagraph 1:\n");
      printf("Longest word  : %s\n", longest1);
      printf("Shortest word : %s\n", shortest1);

      printf("\nParagraph 2:\n");
      printf("Longest word  : %s\n", longest2);
      printf("Shortest word : %s\n", shortest2);

      break;
```

```
case 2:

    removeRepeatedSpaces(paragraph1, modified1);
    removeRepeatedSpaces(paragraph2, modified2);

    printf("\nParagraph 1 after removing repeated spaces:\n");
    printf("%s\n", modified1);

    printf("\nParagraph 2 after removing repeated spaces:\n");
    printf("%s\n", modified2);

    break;


case 3:

    convertToTitleCase(paragraph1, title1);
    convertToTitleCase(paragraph2, title2);

    printf("\nParagraph 1 in Title Case:\n");
    printf("%s\n", title1);

    printf("\nParagraph 2 in Title Case:\n");
    printf("%s\n", title2);

    break;
```

```
        case 4:

            printf("\nParagraph 1:");
            findWordFrequencies(paragraph1);

            printf("\nParagraph 2:");
            findWordFrequencies(paragraph2);

            break;



        case 5:

            printf("Program terminated.\n");
            break;



        default:

            printf("Invalid choice! Please try again.\n");
      }

   } while (choice != 5);

   return 0;
}
```

TEST CASE TABLE:

Test Case	Choice	Expected Output
1	1	P1 longest: programming; shortest: C
2	1	P2 longest: welcome; shortest: to or C
3	2	P1: C programming is very very useful
4	2	P2: hello WORLD welcome to C
5	3	P1: C Programming Is Very Very Useful
6	3	P2: Hello World Welcome To C
7	4	P1: c:1, programming:1, is:1, very:2, useful:1
8	4	P2: hello:1, world:1, welcome:1, to:1, c:1
9	5	Program exits successfully